



PROYECTO FINAL DE SEGURIDAD INFORMÁTICA

Ecosistema de Seguridad Unificado:

Integración de MFA, Infraestructura PKI y Blockchain

Ronal Niño Tineo

Kevin Madrid Neira

Ivan Piñeres Lopez



Tabla de contenido

Tabla de Contenido	¡Error! Marcador no definido.
Resumen Ejecutivo.....	¡Error! Marcador no definido.
1. Problema Identificado y Justificación	¡Error! Marcador no definido.
1.1 El Problema de la Falsificación de Credenciales.....	¡Error! Marcador no definido.
1.2 Justificación del Enfoque Integrado	¡Error! Marcador no definido.
2. Objetivos del Proyecto	¡Error! Marcador no definido.
2.1 Objetivo General	¡Error! Marcador no definido.
2.2 Objetivos Específicos.....	¡Error! Marcador no definido.
3. Arquitectura del Sistema	¡Error! Marcador no definido.
3.1 Componentes Tecnológicos	¡Error! Marcador no definido.
3.2 Flujo de Datos e Interacción.....	¡Error! Marcador no definido.
4. Descripción Técnica de los Módulos	¡Error! Marcador no definido.
4.1 Módulo MFA — Autenticación Multifactor	¡Error! Marcador no definido.
4.2 Módulo PKI — Infraestructura de Clave Pública	¡Error! Marcador no definido.
4.3 Smart Contract y Anclaje Blockchain	¡Error! Marcador no definido.
5. Herramientas Tecnológicas	¡Error! Marcador no definido.
6. Evaluación de Resultados.....	¡Error! Marcador no definido.
6.1 Capacidad como Autoridad Certificadora y Ciclo de Vida.....	¡Error! Marcador no definido.
6.2 Limitaciones Identificadas	¡Error! Marcador no definido.
7. Conclusiones	¡Error! Marcador no definido.
8. Referencias	¡Error! Marcador no definido.



Resumen

El presente informe documenta el diseño e implementación de un prototipo funcional de plataforma web orientada a la gestión y emisión de certificados digitales académicos. El sistema integra tres pilares tecnológicos de seguridad informática moderna: Autenticación Multifactor (MFA), Infraestructura de Clave Pública (PKI) y tecnología Blockchain. Cada componente aborda una capa específica de la seguridad, dando como resultado un esquema de Defensa en Profundidad que protege el acceso, garantiza la identidad y asegura la integridad inmutable de los documentos emitidos.

El prototipo fue desarrollado con Python y Flask como motor del backend, integrando las bibliotecas PyOTP para la validación TOTP, cryptography para la emisión de certificados X.509 con claves RSA de 2048 bits, y Web3.py para la interacción con una red Ethereum local (Ganache). El resultado es un sistema coherente, modular y escalable que puede servir de base para una Autoridad Certificadora (CA) institucional robusta.



1. Problema Identificado y Justificación

1.1 El problema de la Falsificación de Credenciales

Las instituciones educativas y corporativas enfrentan un desafío creciente: el robo de credenciales, los ataques de phishing dirigidos y la falsificación de diplomas y certificados de competencias. En el modelo tradicional, una Autoridad Certificadora (CA) interna protegida únicamente por usuario y contraseña constituye un punto único de falla de alto riesgo. Si un atacante logra comprometer esas credenciales — mediante ingeniería social, fuerza bruta o ataques de diccionario— obtiene la capacidad de emitir certificados digitales falsos de manera indiscriminada, socavando completamente la confianza en el sistema.

1.2 Justificación del Enfoque Integrado

La solución propuesta responde al problema con una arquitectura de triple capa de seguridad:

- MFA: Elimina la dependencia de un factor único (contraseña) para el acceso al sistema de emisión crítica.
- PKI: Provee identidad criptográficamente verificable a cada certificado emitido bajo el estándar X.509.
- Blockchain: Descentraliza y hace inmutable el registro de autenticidad, permitiendo verificación independiente sin depender de una base de datos centralizada.



2. Objetivos

2.1 Objetivo General

Desarrollar un prototipo de plataforma web segura orientada a la gestión y emisión de certificados digitales académicos, integrando controles de acceso mediante Autenticación Multifactor (MFA) y garantizando la integridad de los documentos a través de la inmutabilidad proporcionada por la tecnología Blockchain.

2.2 Objetivo Especificos

1. Diseñar e implementar un esquema de Autenticación Multifactor (MFA) combinando validación tradicional (credenciales) y tokens TOTP para proteger el acceso al sistema de emisión de certificados.
2. Construir un módulo PKI capaz de generar pares de claves RSA de 2048 bits y emitir certificados digitales válidos bajo el estándar X.509.
3. Desarrollar e integrar un Contrato Inteligente (Smart Contract) en Solidity sobre una red Blockchain (Ethereum/Ganache) para anclar y registrar los hashes criptográficos SHA-256 de cada certificado emitido.
4. Evaluar la arquitectura del prototipo como base para una futura Autoridad Certificadora (CA) institucional robusta, identificando sus capacidades y limitaciones en el ciclo de vida de los certificados.



3. Arquitectura del Sistema

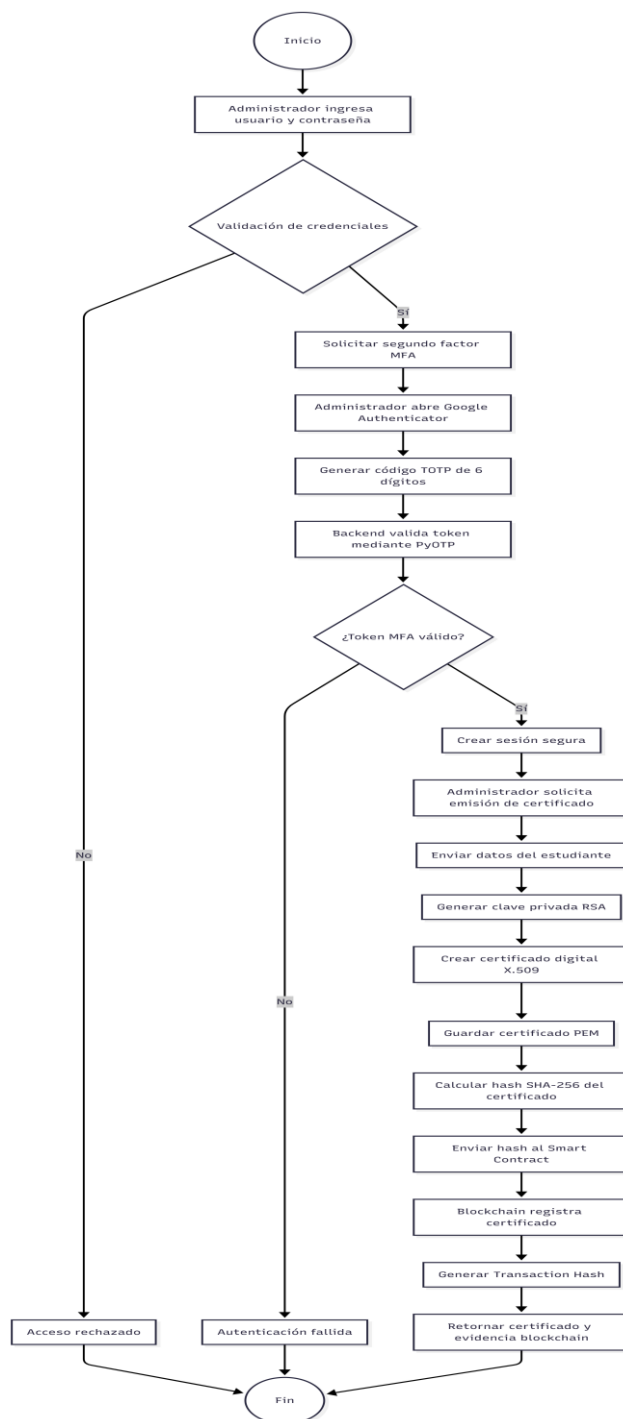
3.1 Componentes Tecnológicos

La arquitectura del prototipo sigue un modelo de capas claramente diferenciadas, donde cada módulo tiene una responsabilidad única y bien definida:

Componente	Descripción
Frontend Web	Interfaz de usuario desarrollada con HTML5, Vanilla CSS con diseño Glassmorphism y JavaScript nativo. Arquitectura Single Page Application (SPA) sin dependencias pesadas.
Backend API	Servidor web implementado en Python con el framework Flask. Gestiona la lógica de negocio, la orquestación de módulos y el enrutamiento de solicitudes HTTP.
Módulo de Identidad	Biblioteca PyOTP para la generación de URIs de aprovisionamiento QR y la validación algorítmica (RFC 6238) de tokens TOTP de 6 dígitos con ventana de 30 segundos.
Módulo PKI	Biblioteca cryptography de Python para la gestión de criptografía asimétrica, generación de claves RSA 2048 bits y estructuración formal de certificados X.509.
Módulo Blockchain	Biblioteca Web3.py conectada a un nodo RPC local de Ganache para la firma y envío de transacciones al Smart Contract desplegado en la red Ethereum simulada.
Smart Contract	Contrato en Solidity con la función registrarHashCertificado(bytes32, string) que almacena el hash SHA-256 y el identificador del estudiante de forma inmutable en la Blockchain.



3.2 Flujo





4. Descripción Técnica de los Módulos

4.1 Modulo MFA – Autenticación Multifactor

La implementación de MFA sigue el estándar TOTP (Time-Based One-Time Password) definido en el RFC 6238. El flujo de aprovisionamiento genera una clave secreta aleatoria por usuario, la cual es codificada en una URI otpauth:// y presentada como código QR para ser escaneada con Google Authenticator.

Durante la verificación, el sistema genera el token válido para la ventana de tiempo actual (30 segundos) utilizando PyOTP y lo compara con el token enviado por el usuario. La biblioteca también admite una tolerancia de ± 1 ventana para compensar desincronizaciones de reloj menores. De esta forma, incluso si un atacante obtiene las credenciales del administrador, no puede acceder al panel de emisión sin poseer físicamente el dispositivo móvil registrado.

Ventaja de seguridad del TOTP

Un token TOTP es válido únicamente durante 30 segundos y es matemáticamente irrepitable. Incluso si un atacante intercepta un token mediante un man-in-the-middle en tiempo real, este expirará antes de que pueda ser reutilizado en otro contexto.

4.2 Modulo PKI – Infraestructura de Clave Publica

El módulo PKI actúa como una Autoridad Certificadora (CA) local. Para cada certificado, el sistema genera un nuevo par de claves RSA de 2048 bits. El certificado X.509 es estructurado con los atributos estándar del sujeto (Common Name, Organization, Country, Email Address) y es firmado digitalmente con la clave privada de la CA, garantizando la autenticidad del documento.

El resultado es un archivo .pem (Privacy Enhanced Mail) que contiene el certificado digital del estudiante en formato Base64. Este archivo puede ser verificado por cualquier sistema compatible con el estándar X.509, como navegadores web, sistemas operativos y bibliotecas criptográficas de uso extendido.

4.3 Smart Contract y Anclaje Blockchain

El Smart Contract fue desarrollado en Solidity y desplegado en una red Ethereum local simulada con Ganache. La función principal registrarHashCertificado(bytes32 hash, string calldata idEstudiante) almacena en el estado inmutable de la Blockchain el hash criptográfico SHA-256 del certificado emitido, asociado al identificador del estudiante.

Una vez que la transacción es minada y confirmada, el Tx Hash devuelto por Ganache sirve como recibo criptográfico de la operación. Cualquier tercero puede calcular de forma independiente el SHA-256 del archivo .pem de un estudiante y consultar el Smart Contract para verificar que ese hash existe en la Blockchain, probando matemáticamente que el certificado fue emitido por esta CA y no ha sido alterado desde su emisión.



5. Herramientas Tecnológicas

Herramienta	Justificación de uso
Python 3.x + Flask	Lenguaje y framework principal. Su ecosistema de librerías de ciberseguridad, la sintaxis expresiva y la rapidez de prototipado lo hacen ideal para proyectos académicos y de investigación.
PyOTP	Implementación de referencia de HOTP/TOTP en Python. Compatible con Google Authenticator y cualquier app conforme al RFC 4226/6238. Permite generación de URIs otpauth:// para QR.
cryptography (Python)	Biblioteca de nivel empresarial para criptografía. Soporta RSA, curvas elípticas, generación X.509, firma digital y gestión de materiales criptográficos con primitivas seguras.
Solidity + Remix IDE	Lenguaje de contratos inteligentes para Ethereum. Remix permite compilar, depurar y desplegar el contrato directamente en Ganache sin configuración adicional.
Web3.py	Librería Python para interactuar con nodos Ethereum. Gestiona la firma de transacciones con la cuenta de Ganache, la invocación de funciones del contrato y la recuperación de Tx Hashes.
Ganache	Simulador de red Ethereum local de Truffle Suite. Provee cuentas con ETH de prueba, minado instantáneo de bloques y una interfaz visual para inspeccionar transacciones.
HTML5 / CSS / JS	Frontend ligero sin frameworks pesados. El diseño Glassmorphism (fondo difuminado con efecto de vidrio) provee una interfaz moderna y profesional con carga mínima.



6. Evaluacion de Resultados

6.1 Capacidad como Autoridad Certificadora y Ciclo de vida

El prototipo implementado cumple de manera excepcional con la fase inicial del ciclo de vida de un certificado: el aprovisionamiento y la emisión. Actúa como una CA local válida porque estructura correctamente los atributos del sujeto y los firma bajo el estándar X.509 reconocido internacionalmente.

La integración con Blockchain añade una capa de verificación descentralizada que elimina la dependencia de una única base de datos tradicional para auditar la validez. Cualquier tercero puede calcular de forma independiente el hash SHA-256 del certificado .pem de un estudiante y consultarlo en el Smart Contract para probar matemáticamente que fue emitido por esta CA y no ha sido alterado. Esto descentraliza la capa de verificación de forma que incluso si la CA institucional desapareciera, los certificados ya emitidos mantendrían su comprobabilidad.

6.2 Limitaciones Identificadas

Limitación	Descripción y mejora propuesta
Gestión de Revocación	El sistema carece de mecanismos formales de revocación (CRL / OCSP). Si la clave de un estudiante es comprometida, no existe actualmente un procedimiento para invalidar el certificado emitido.
Almacenamiento de Clave Privada	La clave privada de la CA reside en software (filesystem). Una CA de producción requiere un Hardware Security Module (HSM) para proteger el material criptográfico maestro.
Red Blockchain de Producción	Ganache es un simulador local. Para producción, el contrato debería desplegarse en una testnet pública (Sepolia) o en una red privada empresarial (Hyperledger Besu).
Gestión de Roles (RBAC)	El sistema actual no implementa un control de acceso basado en roles robusto. Un esquema RBAC permitiría diferenciar permisos entre super-administradores, emisores y auditores.
Trazabilidad de Auditoría	No existe un log inmutable de todos los intentos de acceso y operaciones de emisión. Herramientas como HashiCorp Vault podrían proveer un registro de auditoría completo.



7. Conclusiones

- La implementación de MFA basada en TOTP elimina los vectores de ataque fundamentales asociados al robo de credenciales. Incluso ante una brecha total de la contraseña del administrador, el sistema permanece seguro gracias al segundo factor temporal.
- La convergencia entre PKI tradicional y Blockchain resuelve el problema histórico de la "confianza centralizada" en los sistemas de certificación. El registro inmutable en cadena de bloques facilita la auditoría transparente sin exponer datos sensibles —solo hashes matemáticos— y subsiste independientemente de la disponibilidad de la CA.
- El proyecto valida el principio de Defensa en Profundidad: MFA para el control de acceso, PKI para la identidad y confidencialidad del dato emitido, y Blockchain para la integridad y el no repudio del registro histórico. Ninguna capa es suficiente por sí sola; su fortaleza emerge de la integración.
- La arquitectura modular del prototipo, con responsabilidades claramente separadas (autenticación, emisión, anclaje), facilita la extensión incremental hacia una CA institucional completa, incorporando progresivamente las mejoras identificadas: revocación (CRL/OCSP), HSM, RBAC y logging de auditoría.